Security Features of a Cloud Platform: A Review of Identity and Access Management, Encryption and Data Protection on AWS, with a Comparison to Microsoft Azure and Google Cloud

Name: Jainisha Makwana **Enrollment No:** 20230905090017 **Class:** C

Subject: Cloud Computing (ALA 3) **Submitted to:** Dhaval Chandarana

Abstract

Moving data and applications to the cloud shifts part of the security work to the cloud provider, but not all of it. This review paper studies the security features of Amazon Web Services (AWS), with a focus on three areas: Identity and Access Management (IAM), encryption, and data protection. The AWS features are compared with the equivalent features of Microsoft Azure and Google Cloud. Two real-world incidents, the 2019 Capital One breach and the 2023 Microsoft Storm-0558 intrusion, are used to show how weaknesses appear in practice. The review finds that all three providers encrypt stored data by default, so the larger differences lie in identity models and in how much control customers have over keys. The incidents and industry threat reports show that misconfiguration and weak identity control are more common causes of cloud breaches than failures of the encryption itself. The paper ends with a set of best practices for securing cloud environments.

Keywords: cloud security, IAM, encryption, AWS, shared responsibility, least privilege, data protection

1. Introduction

Cloud computing lets organizations rent computing, storage and networking from a provider instead of owning hardware. This brings speed and flexibility, but it also changes who is responsible for security. The National Institute of Standards and Technology (NIST) advises organizations to plan the security and privacy aspects of a cloud solution before adopting it, and states that accountability for the security and privacy of their data stays with the organization even when a public cloud is used [1].

This paper reviews the built-in security features of AWS because it is one of the most widely used platforms and has detailed public documentation. The review covers three questions:

1. How does the platform control who can access resources (IAM)?
2. How does it protect data with encryption and key management?
3. What other mechanisms protect data and detect misuse?

The paper then compares AWS with Azure and Google Cloud, studies two real incidents, and lists best practices. **Method and scope:** this is a literature and documentation review. It is based on the official documentation of the three providers, NIST guidance, an industry threat report, and published reports about the incidents, all accessed on 7 November 2026. No hands-on penetration testing was done, and cloud features change often, so the details should be re-checked in the current documentation.

2. Foundations of Cloud Security

2.1 The shared responsibility model

AWS describes security as a shared responsibility. AWS is responsible for the security *of* the cloud, which means the infrastructure that runs its services. The customer is responsible for security *in* the cloud. The amount of customer work depends on the services chosen, and customers are responsible for managing their data (including encryption options), classifying their assets, and using IAM tools to apply suitable permissions [2]. In practice this means that a provider can run very secure data centres, but a customer who leaves a storage bucket open or gives a role too many permissions can still lose data.

2.2 The threat landscape

The Cloud Security Alliance (CSA) publishes a survey-based ranking of the main cloud threats. In its 2024 report, the top four were misconfiguration and inadequate change control (ranked first), identity and access management, insecure interfaces and APIs, and an inadequate cloud security strategy [21][22]. The report also notes that traditional concerns about the provider itself had become less important than customer-side weaknesses [21]. This supports the focus of this paper on IAM, encryption and data protection controls, because these are the areas the customer configures.

3. Identity and Access Management (IAM)

IAM is the system that decides who can do what on which resource. In AWS, IAM lets an organization specify who can access which services and resources, and under which conditions [3]. The main building blocks are users, groups, roles and policies. Policies are JSON documents that list the allowed actions on specific resources. Roles are identities that can be assumed by people or by workloads such as a virtual machine.

AWS publishes best practices for IAM, which can be summarized as follows [3]:

- **Use temporary credentials.** Human users should reach AWS through an identity provider and by assuming roles, and workloads should use IAM roles, so that access relies on temporary credentials instead of long-lived access keys. Where long-term access keys are unavoidable, they should be rotated regularly.
- **Require multi-factor authentication (MFA).** If IAM users or the root user are needed, MFA should be required for extra protection.
- **Grant least privilege.** Policies should give only the permissions needed for the task. AWS notes that its managed policies may not be least-privilege for a specific use case, and recommends removing unused users, roles and permissions.
- **Use analysis tools.** IAM Access Analyzer can generate a least-privilege policy from the actions a role actually uses and can validate policies against best practices. Last-accessed information helps find permissions that are no longer needed.
- **Protect the root user.** The root account should be protected with MFA and used only for tasks that need it.

The key idea is that identity is the new security perimeter in the cloud. A compromised credential works from anywhere on the internet, so strong authentication and narrow permissions limit the damage of any single mistake.

4. Encryption Techniques

4.1 Encryption at rest in Amazon S3

Amazon S3, the object storage service, now encrypts every new object by default. Since 5 January 2023, all new object uploads are encrypted automatically with server-side encryption using S3-managed keys (SSE-S3), at no extra cost, and this base level of encryption cannot be turned off [4][6]. SSE-S3 uses 256-bit AES in Galois/Counter Mode [5]. Customers can choose other options when they need more control [5][6]:

- **SSE-KMS and DSSE-KMS:** keys are managed in AWS Key Management Service. KMS adds separate permissions for using the key, which gives an extra layer of control against unauthorized access to the objects [6]. When SSE-KMS is the default, S3 Bucket Keys can reduce the number of requests sent to KMS and lower cost [7].
- **SSE-C:** S3 performs the encryption, but the customer supplies and manages the keys [6].
- **Client-side encryption:** the customer encrypts the data before upload and manages the keys and tools [5].

Changing a bucket's default encryption does not re-encrypt existing objects; S3 Batch Operations can be used to copy them into the new encryption type [4][7].

4.2 Key management with AWS KMS

AWS KMS uses envelope encryption. A data key encrypts the data, and that data key is itself protected under a KMS key, so the data key can be stored safely next to the encrypted data [8]. The KMS keys never leave the service's hardware security modules (HSMs) unencrypted, and AWS documents these HSMs as validated to FIPS 140-3 Security Level 3 [8]. There are three kinds of keys: customer managed keys created by the organization, AWS managed keys created by AWS services in the customer's account, and AWS owned keys [9]. AWS recommends customer managed keys that are created and stored in the KMS HSMs [9]. Because KMS has its own permissions for using a key, an organization can control who may use a key separately from who may read the stored objects [6].

4.3 Encryption in transit

Data protection covers data while it moves to and from a service as well as data at rest [5]. Customers should therefore require encrypted connections for their own applications and for access to storage. This review concentrates on encryption at rest, because that is where the providers differ most in how much key control they offer.

5. Data Protection Mechanisms

Encryption alone does not protect data if access is too open or if misuse goes unnoticed. AWS therefore provides prevention, detection and recovery controls around storage:

- **Secure defaults.** All newly created S3 buckets have Block Public Access enabled, access control lists (ACLs) disabled, and all new objects encrypted [10].

- **Block Public Access.** This setting works at the account level and on individual buckets, including future ones. AWS recommends enabling it and using Organizations service control policies (SCPs) so that the setting cannot be changed [11].
- **Logging and threat detection.** Amazon GuardDuty S3 Protection monitors AWS CloudTrail data events for S3, which include object-level operations, to detect risks such as data exfiltration and destruction, and it generates a finding when it detects a potential threat [12].
- **Sensitive-data discovery.** Amazon Macie helps find sensitive data outside designated areas [11]. Other services such as AWS Security Hub, AWS Config and Trusted Advisor help review bucket configuration [10].
- **Recovery.** S3 Versioning, Replication and Object Lock add protection against deletion, corruption and ransomware [13].

These layers follow a defense-in-depth idea: if one safeguard fails, another should still reduce the damage [13].

6. Comparative Analysis: AWS, Azure and Google Cloud

Azure and Google Cloud provide features in the same three areas. The table compares the main points found in the official documentation.

Area	AWS	Microsoft Azure	Google Cloud
Identity service	IAM with users, groups, roles and JSON policies [3]	Microsoft Entra ID for identity, with Azure role-based access control (RBAC) for resources [16]	IAM with principals, roles and allow policies attached to resources [18]
Permission model	Policies on identities and resources; Organizations SCPs as guardrails [3][11]	Built-in and custom roles, aimed at granting minimum permissions [16]	Allow policies are inherited down the resource hierarchy (organization, folders, projects, resources) [18]
Strong sign-in	MFA required for IAM users and root [3]	Conditional Access can enforce MFA by user, location, device and application (Entra ID P1/P2 licences) [16]	Not detailed in the sources reviewed
Temporary privilege	Roles with temporary credentials [3]	Privileged Identity Management for just-in-time access [17]	Privileged Access Manager for temporary, auditable access [18]
Default encryption at rest	SSE-S3 (AES-256) for all new S3 objects, cannot be disabled [4][5][6]	Storage encryption with 256-bit AES for all storage accounts, cannot be disabled [15]	All customer content encrypted at rest by default, with AES-256 at the storage layer [19]
Customer-controlled	SSE-KMS, DSSE-KMS, SSE-C, client-	Customer-managed keys in Key Vault or Managed HSM;	Customer-managed keys (CMEK) in Cloud KMS:

Area	AWS	Microsoft Azure	Google Cloud
keys	side [5][6]	customer-provided keys for Blob Storage [15]	server-side envelope encryption with 256-bit AES-GCM keys [20]
Key protection	KMS keys stay inside FIPS 140-3 Level 3 HSMs [8]	Managed HSM gives a customer-owned security domain that Microsoft cannot access [14]	Key material never leaves the Cloud KMS system boundary [20]
Default keys	Owned and managed by AWS (S3-managed keys) [6]	Microsoft-managed keys; with customer-managed keys, rotation is the customer's duty [15]	Google owns and manages the default keys [19]

Observations.

1. **Encryption is a common baseline.** All three providers encrypt stored data by default with AES-256 [4][15][19]. On AWS and Azure the customer cannot switch this off [4][15], so the choice between providers rarely depends on whether data is encrypted.
2. **Key control is the real option.** Each provider offers customer-managed keys. The extra control comes with extra duties: for example, with Azure customer-managed keys the customer becomes responsible for key rotation [15]. Customers with regulatory needs should choose these options deliberately.
3. **Identity models differ.** Google Cloud's inherited allow policies reward careful design of the resource hierarchy [18]. Azure combines Entra ID sign-in controls with RBAC [16]. AWS centres on roles and policies, with Organizations guardrails [3][11].
4. **Monitoring services exist on all three platforms.** This review describes the AWS services in detail [10][12], and the same layered approach (logging, threat detection, sensitive-data discovery) should be applied on any platform.

7. Real-World Scenarios

7.1 Capital One breach (2019)

In 2019, personal information of about 100 million people in the United States and about 6 million in Canada was taken from data stored in AWS by a former AWS employee [26]. According to the Department of Justice complaint as reported, a firewall misconfiguration allowed commands to reach and be executed by a server, which gave the attacker access to data folders (buckets) on AWS [25]. In steps, the attacker obtained credentials for a web application firewall role, listed the contents of the storage, and then copied the data [24]. Commentators quoted at the time noted that the infiltrator got in through the firewall that was running on a Capital One server, and that blaming Amazon for the breach was misinformed [25].

The consequences were large. Capital One was fined 80 million US dollars by the US Office of the Comptroller of the Currency [27] and settled customer lawsuits for 190 million dollars [23]. The attacker was later convicted [23].

Lessons.

- *Least privilege:* the role used in the attack could reach far more data than a firewall component needs.
- *Secure configuration and change control:* a single misconfigured component started the chain, which matches the CSA ranking of misconfiguration as the top threat [21].
- *Limit access to the instance metadata service* so that a server cannot be tricked into revealing role credentials [26].
- *Detection:* unusual access and data transfers should raise alarms quickly, which is the purpose of logging and threat detection services such as CloudTrail and GuardDuty [12][26].
- *Encryption was not enough:* the credentials authorized the reading of data, so encryption at rest could not stop the access. Strong IAM is what limits who can read decrypted data.

7.2 Microsoft Exchange Online intrusion by Storm-0558 (2023)

In 2023, a threat actor tracked as Storm-0558 accessed the email of a broad range of victims by forging authentication tokens with a stolen Microsoft account (MSA) signing key from 2016 [28]. The key was meant to sign tokens for consumer accounts only, but a flaw in token validation let it work for enterprise email as well [28]. Reports state that the attackers reached the mailboxes of 22 organizations [30]. The US Cyber Safety Review Board (CSRB) concluded that the intrusion was preventable [29]. It was also criticized that a key from 2016 was still valid in 2023 [30], and the board noted that Microsoft still could not show how the key was stolen [28].

Lessons.

- *Key lifecycle management matters:* old keys should be rotated and retired, since a single signing key can allow forged identities without any password [30].
- *Validation controls:* systems must check that a credential is valid for the service being accessed [28].
- *Provider-side risks exist:* customers cannot rotate a provider's internal keys, which is why logging, MFA and monitoring on the customer side, and honest incident reporting by the provider, remain important.

7.3 Applying the controls: a small web application on AWS

Consider a student web application running on an EC2 virtual machine that stores user uploads in S3. A secure design would apply the findings of this review as follows:

1. Attach an IAM role to the virtual machine, allowing only listing, reading and writing for the one bucket, so the application needs no access keys in its code [3].
2. Keep S3 Block Public Access on, and share files through temporary signed links instead of making the bucket public [11].
3. Keep default encryption on, and use a customer managed KMS key if the files are sensitive [6][9].
4. Use MFA on the root and administrator accounts, and use the root account only when required [3].
5. Turn on logging and threat detection, and review the findings regularly [12].
6. Close unused ports in the firewall rules (security group) and review the rules after deployment.

This design limits what an attacker could do even if one control fails, which is the lesson of both incidents.

8. Best Practices for Securing Cloud Environments

Area	Practice	Basis
Governance	Plan security before adopting the cloud, keep accountability for data, and use a risk-based approach	[1][2]
Governance	Use change control and automated configuration checks, because misconfiguration is the top threat	[21][22]
Identity	Use roles and temporary credentials instead of long-term keys; require MFA; protect the root user	[3]
Identity	Grant least privilege, review unused permissions, and use Access Analyzer	[3]
Identity	Use just-in-time elevation for privileged work (for example PIM on Azure, PAM on Google Cloud)	[17][18]
Encryption	Keep default encryption on; use customer managed keys for sensitive or regulated data and understand the extra duties	[6][9][15]
Encryption	Separate permission to use a key from permission to read the data; rotate and retire keys	[6][15][30]
Data protection	Keep Block Public Access on and enforce it with organization-level policies	[11]
Data protection	Use versioning, replication and object lock for recovery	[13]
Monitoring	Enable activity logging and threat detection, and discover sensitive data	[11][12]
Architecture	Follow a zero-trust approach: verify continuously and give least-privilege access	[31]

9. Limitations

This review relies on public documentation and press and government reports. Provider documentation describes features as the provider intends them, and it can change; the claims were accurate as of the access date. The comparison covers selected features and is not a complete audit of any platform. Costs, compliance certifications and performance were outside the scope. Details of the incidents come from published reports and may be updated as more information becomes available.

10. Conclusion

This review examined the identity, encryption and data protection features of AWS and compared them with Azure and Google Cloud. All three platforms provide strong encryption by default and offer customer-managed keys for organizations that need more control. The most important differences are in identity models and in the responsibilities that come with key control. The Capital One and Storm-0558 incidents show that failures usually occur through excessive permissions, misconfiguration and weak key or credential management, not through breaking the encryption. Cloud security is therefore a shared effort: the provider secures the platform, and the customer must configure identity, keys and monitoring carefully and keep them under review.

References

All web pages were accessed on 7 November 2026.

[1] T. Grance and W. Jansen, *Guidelines on Security and Privacy in Public Cloud Computing*, NIST SP 800-144, 2011. https://doi.org/10.6028/NIST.SP.800-144

[2] Amazon Web Services, “Shared Responsibility Model.” https://aws.amazon.com/compliance/shared-responsibility-model/

[3] Amazon Web Services, “AWS Identity and Access Management best practices.” https://aws.amazon.com/iam/resources/best-practices

[4] Amazon Web Services, “Default encryption FAQ,” Amazon S3 User Guide. https://docs.aws.amazon.com/AmazonS3/latest/userguide/default-encryption-faq.html

[5] Amazon Web Services, “Using server-side encryption with Amazon S3 managed keys (SSE-S3),” Amazon S3 User Guide. https://docs.aws.amazon.com/us_en/AmazonS3/latest/userguide/UsingServerSideEncryption.html

[6] S. Stormacq, “Amazon S3 Encrypts New Objects By Default,” AWS News Blog. https://aws.amazon.com/jp/blogs/aws/amazon-s3-encrypts-new-objects-by-default/

[7] Amazon Web Services, “Setting default server-side encryption behavior for Amazon S3 buckets,” Amazon S3 User Guide. https://docs.aws.amazon.com/en_br/AmazonS3/latest/userguide/bucket-encryption.html

[8] Amazon Web Services, “AWS KMS cryptography essentials,” AWS Key Management Service Developer Guide. https://docs.aws.amazon.com/kms/latest/developerguide/kms-cryptography.html

[9] Amazon Web Services, “AWS Key Management Service FAQs.” https://aws.amazon.com/kms/faqs/

[10] Amazon Web Services, “Find public S3 buckets in your AWS account,” AWS Storage Blog. https://aws.amazon.com/blogs/storage/find-public-s3-buckets-in-your-aws-account